

Codenza

Online Judge Platform — High Level Design

Geetanjali Reddy

Overview

Codenza is a full-stack MERN application that provides a secure and efficient environment for solving Data Structures and Algorithms (DSA) problems. Users can browse coding questions, write solutions in an integrated code editor, and submit them for automated evaluation. Submitted programs are executed inside isolated containers against hidden test cases, generating verdicts such as Accepted, Wrong Answer, Time Limit Exceeded, and Runtime Error.

The platform includes JWT-based authentication, problem management, submission history, user profiles, and AI-powered features such as hint generation, code review, and error explanation. An asynchronous job queue ensures scalable code execution, while the Interview Readiness Score analyzes users' performance across different topics, identifies strengths and weaknesses, and recommends personalized learning paths along with suitable interview target companies.

1. Database Design

Hidden test cases are stored in a dedicated collection rather than embedded inside Problems, so that grading data stays isolated from problem metadata, is easier to access-control, and is never accidentally pulled into AI prompt context (see Section 5 — Security).

users

Field	Description
_id, name, email, passwordHash, role	Core identity and access control fields
avatar	Profile image URL
solvedProblems, totalSubmissions	Aggregate stats for profile display
createdAt	Account creation timestamp

problems

Field	Description
_id, title, statement, difficulty, tags, constraints	Problem metadata shown to users
sampleTests, boilerplate	Public example tests and starter code per language
timeLimitMs, memoryLimitMb	Execution limits used by the judge
createdBy, createdAt	Authoring metadata

hiddenTests (new — separated from problems)

Field	Description
_id, problemId	Reference back to the parent problem
input, expectedOutput	Grading data, never exposed via any public API
visibility: "internal"	Enforced at the schema/API layer — excluded from all client-facing serializers and from any AI prompt payload

submissions

Field	Description
_id, userId, problemId, language, code	Submission identity and payload
verdict, runtimeMs, memoryKb	Judge output
passedTestCases, totalTestCases, submittedAt	Result summary and timestamp

ai_interactions

Field	Description
_id, userId, problemId	Context for the interaction
type (Hint / Review / Error Explanation)	Which AI feature was used
prompt, response	Sanitized request sent to the LLM and its reply — sanitization strips hidden test data and any injected instructions (see Section 5)
createdAt	Timestamp

2. Web Server Design

a) Frontend

Built with React and Tailwind CSS.

Screen 1 — Authentication

- Login
- Sign Up
- Forgot Password

Screen 2 — Home

- List of coding problems
- Search and filter by difficulty / tags
- User profile shortcut

Screen 3 — Problem

- Problem statement, constraints, sample input/output

- Language selector and code editor
- Run Code / Submit Code
- Verdict panel
- AI Hint / Code Review

Screen 4 — Submission History

- Previous submissions with verdict, language, runtime, memory usage, and timestamp

Screen 5 — User Profile

- Personal information, solved problems, total submissions
- Coding DNA Profile and AI learning insights

All frontend requests communicate with the backend through secured REST APIs using JWT authentication.

b) Backend

The backend follows an MVC architecture and exposes REST APIs for authentication, problem management, submissions, submission history, profile management, and admin operations (expanded from the original design, which only covered auth, problems, and submission creation).

Authentication

- POST /auth/register
- POST /auth/login
- GET /auth/profile
- **New:** PUT /auth/profile — update profile / avatar
- **New:** POST /auth/refresh-token — rotate JWT without re-login

Problem APIs

- GET /problems — list of available problems with title, difficulty, and tags
- GET /problems/:id — full statement, constraints, sample tests, boilerplate
- **New:** POST /problems — create problem (admin only)
- **New:** PUT /problems/:id — edit problem (admin only)
- **New:** DELETE /problems/:id — remove problem (admin only)

Submission APIs

- POST /submissions — create a new submission, enqueued for evaluation
- **New:** GET /submissions/:userId — paginated submission history for a user
- **New:** GET /submissions/:id — single submission detail with verdict breakdown

AI & Readiness APIs

- **New:** POST /ai/hint, POST /ai/review, POST /ai/explain-error
- **New:** GET /readiness/:userId — cached readiness score and recommendations

3. Execution Pipeline

Expanded from a single "Docker execution" step into an explicit pipeline, so the design communicates how concurrency and isolation are actually handled:

1. Submission — user submits code via POST /submissions; the API persists the submission with status Queued and returns immediately (non-blocking).
2. Queue — the submission is pushed onto a job queue (BullMQ on Redis, or RabbitMQ) keyed by language, enabling horizontal scaling of workers and back-pressure control under load.
3. Worker — a pool of worker processes pulls jobs from the queue, one job per available slot, and prepares the language-specific execution environment.
4. Docker — each job runs inside a short-lived, resource-capped container (CPU, memory, network disabled) scoped to a single submission; the container is destroyed immediately after execution.
5. Verdict — the worker compares actual output against hidden test cases, writes the verdict and runtime/memory stats back to the submission record, and publishes a result event (WebSocket / polling) so the frontend updates live.

This queue-worker separation is what allows the platform to handle concurrent submissions during traffic spikes (e.g., contests) without overloading a single execution host.

Verdict Conditions

Verdict	Condition
Accepted	Output matches expected output on every test case
Wrong Answer	Output differs from expected on some test case
Time Limit Exceeded	Runtime exceeds the problem's time limit
Memory Limit Exceeded	Memory usage exceeds the problem's memory limit
Runtime Error	Program crashes or exits with a non-zero code
Compilation Error	Code fails to compile — no test case is run

4. AI Integration

- Hint Generation — fetched for the problem currently open, using its statement and the user's code, without revealing the full solution. (Gemini API)
- Code Review — runs after an Accepted verdict; comments on approach and complexity.
- Error Explanation — translates a raw compiler/runtime error into plain language.

5. Scalability & Security Considerations

Added section — addresses concurrency handling and the specific risks of routing user-submitted code and AI prompts through an automated pipeline.

Scalability

- Redis-backed BullMQ (or RabbitMQ) queue decouples submission intake from execution, so traffic spikes queue up instead of overwhelming workers.
- Worker pool scales horizontally; workers are stateless and can be added/removed based on queue depth.
- Readiness Score results are cached (Redis) and only recomputed when new submissions meaningfully change a user's performance, avoiding repeated AI calls per profile view.

Security

- Hidden test cases live in a separate collection, excluded from every client-facing API response and from any payload sent to the LLM.
- Prompt injection mitigation — user code and problem statements are sanitized before being interpolated into an AI prompt; the LLM is instructed via a fixed system role, and its output is treated as untrusted text (escaped, never executed, never used to alter application logic).
- Each submission executes in a network-disabled, resource-capped container with no access to the host filesystem, other users' data, or the hidden test store.
- JWT tokens are short-lived with refresh rotation; admin-only routes (problem CRUD) are protected by role-based middleware.

6. Interview Readiness Score

Unlike traditional online judges that only display the number of solved problems, this platform continuously analyzes a user's submission history to estimate their interview readiness. Based on problem-solving patterns, topic coverage, acceptance rate, execution efficiency, and coding consistency, the AI generates a personalized readiness report and recommends areas that require improvement.

Scoring Factors & Approximate Weighting

Weights below are approximate and tunable; they are surfaced here so the score is not a black box.

Factor	Approx. Weight	What it captures
Topic-wise coverage	25%	Breadth across Arrays, Trees, Graphs, DP, Greedy, etc.
Acceptance rate (first attempt)	20%	Correctness without relying on repeated tries
Execution efficiency	15%	Runtime / memory relative to problem limits
Difficulty performance	20%	Success split across Easy / Medium / Hard
Coding consistency over time	10%	Regularity of practice, not just volume
AI code review feedback	10%	Qualitative signal on approach and complexity

Overall Score is computed as a weighted combination of the above — not a direct average of individual topic scores.

Sample Report

Metric	Value
Overall Score	68%
Arrays	95%
Strings	82%
Trees	46%
Graphs	48%
Dynamic Programming	49%
Greedy	54%

Recommendation

Improve Dynamic Programming and Graph algorithms to increase your readiness score.

Ready for

- Deloitte
- Infosys

Needs Improvement

- Google (Dynamic Programming)
- Atlassian (Graphs)

Suggested Learning Path

6. Number of Islands
7. Course Schedule
8. Word Ladder
9. Edit Distance

Backend Implementation

After every accepted submission, the backend updates the user's learning profile using MongoDB aggregation pipelines. The AI service analyzes submission history, runtime statistics, topic distribution, and code review feedback to generate an updated readiness score and personalized recommendations. The computed results are cached to reduce repeated AI requests and refreshed whenever new submissions significantly change the user's performance.